# Classes, Objects and Constructors

9 min read  Updated 28 Jul 2026  Topic 6 of 65

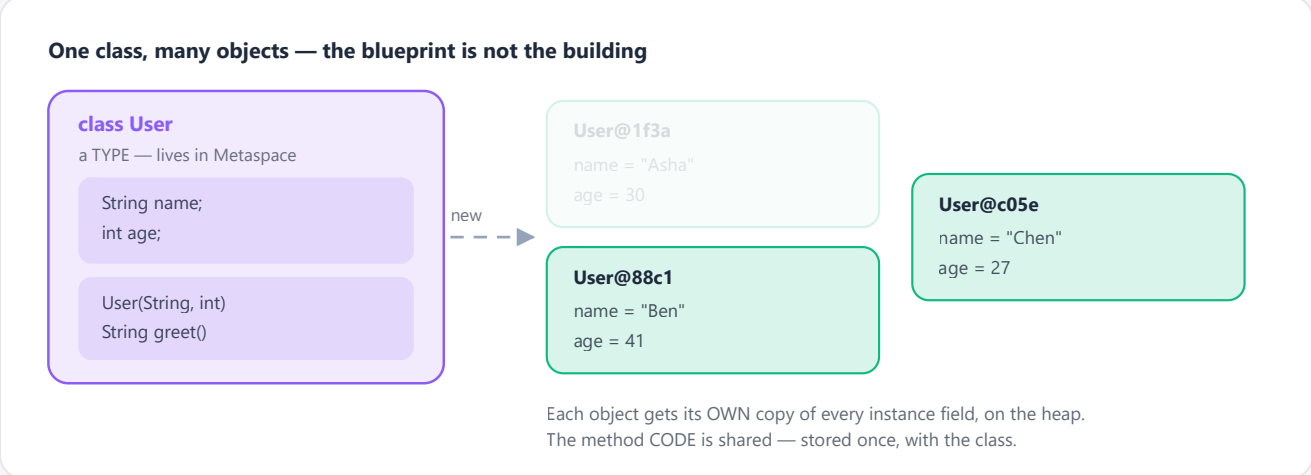
Bookmark  Mark done

## Why it matters

Every Java program you will ever write is classes and objects, so it is tempting to skip this as revision. Do not. The specific things interviewers probe here — what a constructor is *not*, why constructor chaining exists, why calling an overridable method from a constructor is a bug — are the ones that separate someone who has typed `new` ten thousand times from someone who knows what it does. They also underpin Chapter 11: Spring's whole dependency-injection model is an argument about how objects should be constructed.

## The concept, simply

A **class** is a type: a description of what a thing has and what it can do. An **object** is one concrete instance of that description, living on the heap. The class is written once; objects are created as often as you like, and each gets its own copy of the instance fields while sharing one copy of the method code.




The class is the blueprint and lives in Metaspace. Each object is a separate allocation on the heap with its own field values — but the method bytecode is stored once, with the class.

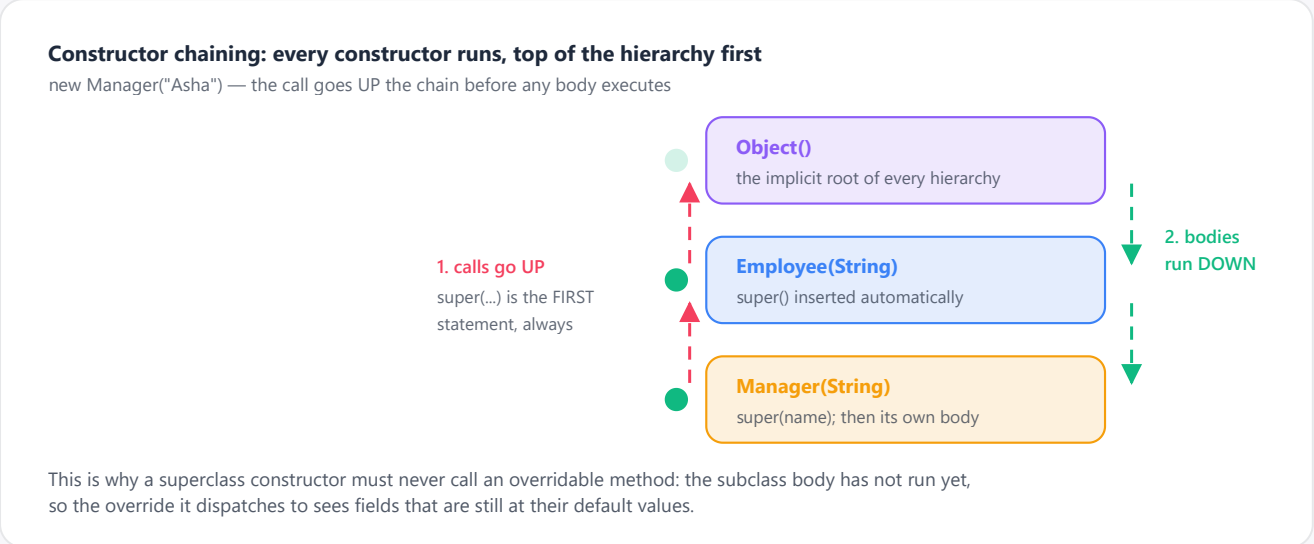
A **constructor** is the code that runs to bring a new object into a valid state. It looks like a method but is not one: it has no return type, its name must exactly match the class, and it cannot be called on an existing object. Its single job is to establish the invariants the rest of the class then relies on.

## How it works

`new User("Asha", 30)` is four steps, in order:

1. **Allocate** — the JVM reserves heap space for the object's fields plus its header, and zeroes it. Every field is now `0`, `false` or `null`.
2. **Chain upward** — the constructor's first act is to call a superclass constructor, all the way to `Object`.
3. **Initialise** — instance field initialisers and instance blocks run, then the constructor body.
4. **Return the reference** — `new` evaluates to the address of the finished object.

Step 2 is the one worth internalising. **Every constructor's first statement is a call to another constructor** — either `this( ... )` for a sibling in the same class, or `super( ... )` for the parent. If you write neither, the compiler silently inserts `super()`. This is why an entire chain of constructors runs on a single `new`.




Calls travel up the hierarchy first; bodies then execute downward. By the time Manager's body runs, every ancestor has finished initialising — which is exactly the guarantee it depends on.

`super()` does not compile. Second, the parent's body runs *before* the child's, which is why calling an overridable method from a constructor is dangerous: the override executes against a half-built child.

The **default constructor** has a rule people get wrong: the compiler generates a no-argument constructor *only if you declare no constructor at all*. Write any constructor and the free one disappears — which is why adding `User(String name)` to a class can break code that called `new User()`.

## Code examples

JAVA

Copy

```java
public class User {
    private final String name;      // final → must be assigned exactly onc
    private int age;
    private final List<String> roles = new ArrayList<>();   // field initi

    // Primary constructor — establishes every invariant in one place.
    public User(String name, int age) {
        if (name == null || name.isBlank()) throw new IllegalArgumentExcep
        if (age < 0) throw new IllegalArgumentException("age must be ≥ 0"
        this.name = name;           // "this." disambiguates field from par
        this.age = age;
    }

    // Convenience constructor — delegates rather than duplicating validat
    public User(String name) {
        this(name, 0);              // must be the FIRST statement
    }
}
```

Delegating with `this( ... )` rather than copy-pasting validation is the habit interviewers look for: it means the invariants are enforced in exactly one place, no matter which constructor is used.

The classic constructor bug, which is worth being able to recognise on sight:

```
class Parent {
    Parent() {
        init();                    // ✗ calls the OVERRIDE, on a half-buil
    }
    void init() { System.out.println("Parent.init"); }
}

class Child extends Parent {
    private String value = "ready";

    @Override
    void init() {
        System.out.println("Child.init, value = " + value);
    }
}

new Child();
// Parent.init  is never printed — Child.init runs instead
// Child.init, value = null
//
// Why: the parent constructor runs BEFORE Child's field initialisers, so
// "value" is still null when the override executes. Never call an
// overridable method from a constructor.
```

And the disappearing default constructor:

JAVA Copy

```
class A { }                        // compiler adds A() for you
new A();                           // ✓ compiles

class B { B(String s) { } }        // you declared one → no free A()
// new B();                        // ✗ "constructor B in class B cannot b
//                                       applied to given types"
```

## Under the hood

In bytecode a constructor is a method named `<init>`, and `new` compiles into two separate instructions — `new` (allocate and zero) followed by `invokespecial <init>` (run the constructor). Those being separate is the reason a partially-constructed object can be observed at all: between them, the object exists with every field at its default value. It is also why `this` escaping a constructor — registering a listener, starting a thread, publishing to a static registry — is a genuine concurrency hazard. Another thread can reach an object whose `final` fields are not yet visible to it. Chapter 8 returns to this as *unsafe publication*.

The **object header** is worth knowing for memory questions: every object carries roughly 12–16 bytes of header (a mark word for hash code, GC age and lock state, plus a class pointer), padded to an 8-byte boundary. So an object with a single `int` field costs about 16 bytes, not 4. That overhead is exactly the argument for primitives in Chapter 1 and for records in Chapter 10.

## Common mistakes

- **Giving a constructor a return type.** `void User() { }` compiles — as an ordinary method that is never called. A silent, maddening bug.
- **Calling an overridable method from a constructor.** The override runs against uninitialised subclass state. Make such methods `private`, `static` or `final`.
- **Duplicating validation across constructors.** Delegate with `this( ... )` so the rules live once.
- **Assuming the default constructor is always there.** Declaring any constructor removes it.
- **Letting `this` escape.** Publishing a reference before the constructor finishes exposes a half-built object.
- **Telescoping constructors.** Six overloads differing by argument count is a smell — use a builder (Chapter 14) or a record (Chapter 10).

## Interview lens

> **Q: "What is a constructor, and how does it differ from a method?"**
> A: It is the code that initialises a newly allocated object. It has no return type, its

but at the language level it is a distinct construct.

**Q: "Can a constructor be private?"**

A: Yes, and it is a deliberate design tool. A private constructor prevents outside instantiation — used by singletons, by static utility classes so nobody can instantiate them, and by classes that expose static factory methods like `Integer.valueOf` so the class controls its own creation.

**Q: "What is constructor chaining?"**

A: Every constructor begins by invoking another one — `this( ... )` to delegate within the class, or `super( ... )` to the parent — and the compiler inserts `super()` if you write neither. So one `new` runs a chain of constructors from `Object` downward, guaranteeing that by the time a class's body executes, all its inherited state is already initialised.

**Q: "Why must `this()` or `super()` be the first statement?"**

A: Because otherwise you could observe or use inherited state before the superclass had initialised it — the exact bug the chaining rule exists to prevent. (Java 22 relaxes this slightly to allow statements that do not reference the instance, but the guarantee is unchanged.)

**Q: "Are constructors inherited?"**

A: No. A subclass does not inherit its parent's constructors — it must declare its own and call `super( ... )`. This is why adding a required argument to a parent constructor breaks every subclass at compile time, which is usually what you want.

A: The object is never successfully constructed and the reference is never assigned; the partially built object becomes garbage. That is exactly why validation belongs in the constructor — failing there means an invalid instance cannot exist. The caveat is that if `this` escaped before the throw, something may still hold a reference to a broken object.

## Cheat-card

- Class = type (Metaspace). Object = instance (heap). Fields per object; method code shared.
- Constructor: **no return type**, name = class, **not inherited**, compiled as `<init>`.
- First statement is always `this( ... )` or `super( ... )` — inserted implicitly if absent.
- Order: allocate/zero → chain up → field initialisers → constructor body.
- **Default constructor disappears** the moment you declare any constructor.
- Never call an **overridable** method from a constructor — the override sees uninitialised child fields.
- Never let `this` escape a constructor — unsafe publication.
- Object header ≈ 12–16 bytes; an object with one `int` costs ~16, not 4.

QUICK CHECK — no marks, just for you.

1. What is the first statement of every Java constructor?

- ◯ A call to this(...) or super(...) — inserted implicitly as super() if you write neither
- ◯ A null check on the parameters
- ◯ Nothing is required before the constructor body
- ◯ The first field assignment

2. A Parent constructor calls an overridable method init(). Child overrides init() and reads its own field 'value', initialised to "ready". What does the override see?

- A compile error — a constructor cannot call an overridable method
- null, because the parent constructor runs BEFORE the child's field initialisers
- The parent's version of init() runs instead, so the field is irrelevant
- "ready", because field initialisers run before any constructor

Check answer

3. You add a constructor User(String name) to a class that previously had none. What breaks?

- Serialization only
- Nothing — the compiler still provides a no-arg constructor as well
- Existing calls to new User(), because the default constructor disappears once you declare any constructor
- All subclasses, because constructors are inherited

Check answer

‹ Previous
Pass-by-Value: Java's Most Misunderstood Rule

Next ›
Inheritance, Overriding and Polymorphism

Java Interview Kit

LEARN MORE

ARTICLES & DOCS

Classes and objects — The Java Tutorials
docs.oracle.com

Class declarations (JLS §8)
docs.oracle.com

VIDEOS



"classes objects constructors" — video explainers
YouTube



classes objects constructors interview questions
YouTube

Hand-picked references for this lesson.